DSA 8070 R Session 10: Canonical Correlation Analysis

Whitney Huang, Clemson University

Contents

Load and Explore the Data 1
Examine the Two Variable Sets 3
Canonical Correlation Analysis 5
Test the Canonical Correlations 5
Canonical Scores and Loadings 6
Verify CCA Using Eigenvalue Decomposition 9
Verify the Canonical Loadings 11

We use an employee dataset to examine the relationship between two sets of variables:

- **Sales performance (X):** Growth, Profit, and New.
- **Intelligence (Y):** Creativity, Mechanical, Abstract, and Math.

Our goal is to identify linear combinations of the two sets that are maximally correlated.

Load and Explore the Data

```
library(GGally)
library(CCA)
library(CCP)
library(expm)

sales_data <- read.table("sales.txt")
colnames(sales_data) <- c(
  "Growth", "Profit", "New",
  "Creativity", "Mechanical", "Abstract", "Math")

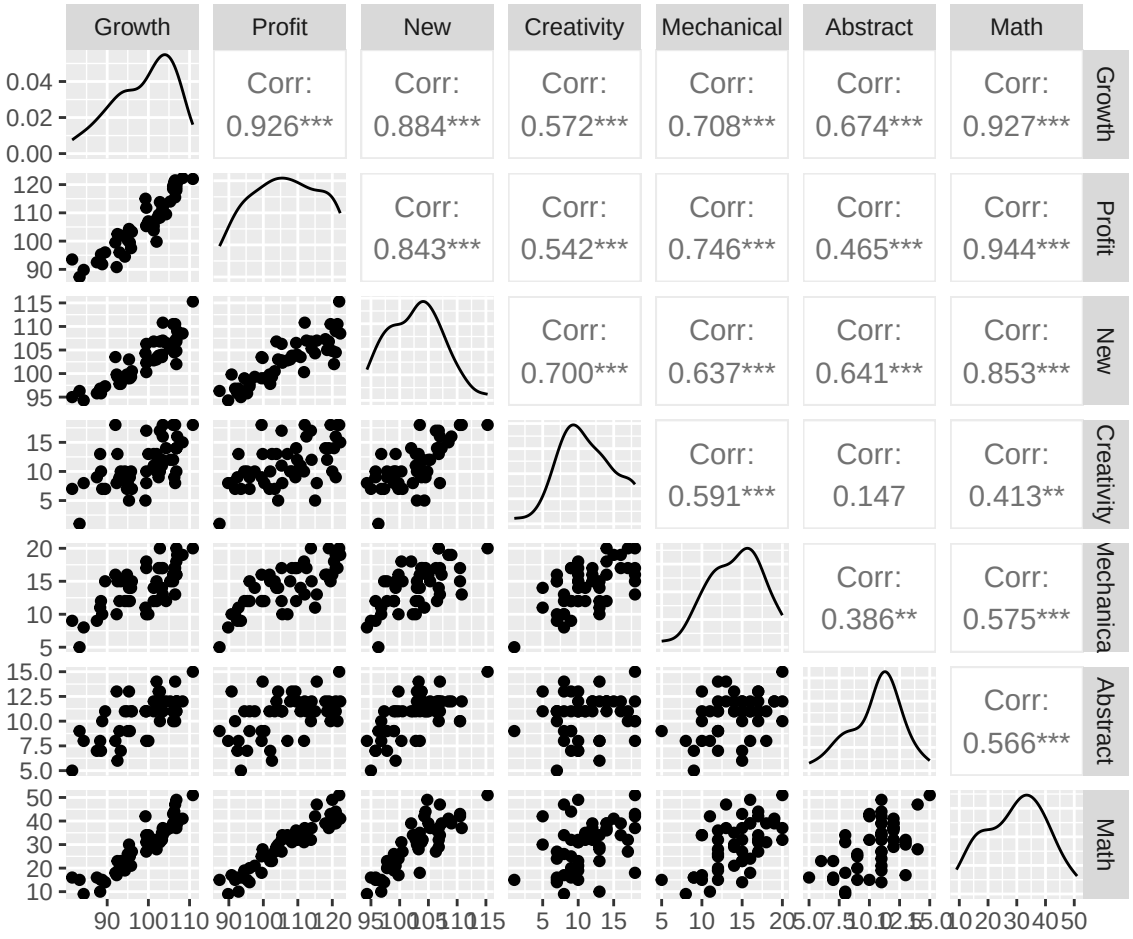
summary(sales_data)
```

```
##      Growth          Profit           New          Creativity
##  Min.   : 81.50   Min.   : 87.3   Min.   : 94.30   Min.   : 1.00
##  1st Qu.: 93.55   1st Qu.: 99.5   1st Qu.: 99.08   1st Qu.: 8.25
##  Median :100.65   Median :106.2   Median :103.15   Median :10.00
##  Mean   : 98.84   Mean   :106.6   Mean   :102.81   Mean   :11.22
##  3rd Qu.:105.05   3rd Qu.:114.8   3rd Qu.:106.45   3rd Qu.:14.00
##  Max.   :110.80   Max.   :122.3   Max.   :115.30   Max.   :18.00
```

```
##    Mechanical       Abstract          Math
##  Min.   : 5.00   Min.   : 5.00   Min.   : 9.00
##  1st Qu.:12.00   1st Qu.: 9.00   1st Qu.:21.50
##  Median :15.00   Median :11.00   Median :31.50
##  Mean   :14.18   Mean   :10.56   Mean   :29.76
##  3rd Qu.:17.00   3rd Qu.:12.00   3rd Qu.:37.00
##  Max.   :20.00   Max.   :15.00   Max.   :51.00
```

Examine the pairwise relationships among the seven variables.

```
ggpairs(sales_data, progress = F)
```




```
round(cor(sales_data), 2)
```

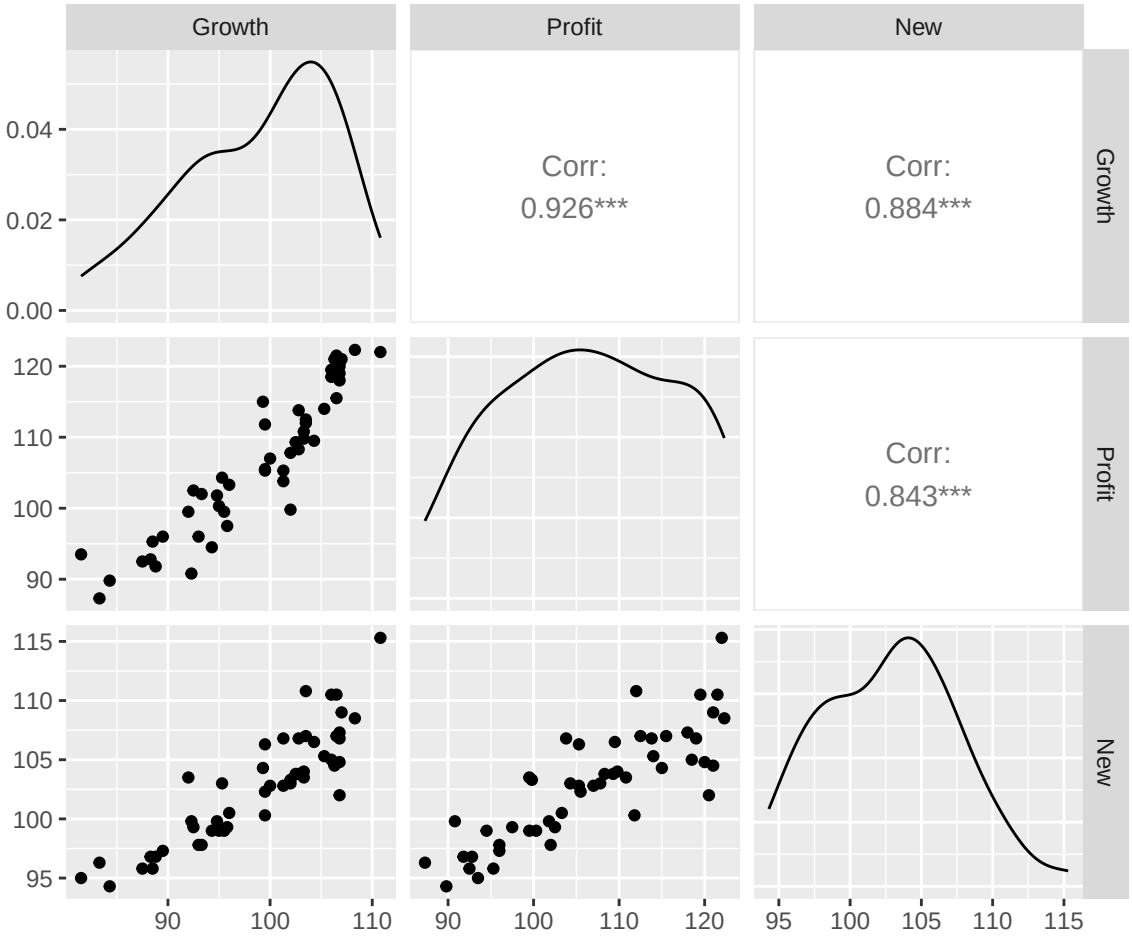
```
##             Growth Profit  New Creativity Mechanical Abstract Math
## Growth        1.00   0.93 0.88       0.57       0.71     0.67 0.93
## Profit        0.93   1.00 0.84       0.54       0.75     0.47 0.94
## New           0.88   0.84 1.00       0.70       0.64     0.64 0.85
## Creativity    0.57   0.54 0.70       1.00       0.59     0.15 0.41
## Mechanical    0.71   0.75 0.64       0.59       1.00     0.39 0.57
## Abstract      0.67   0.47 0.64       0.15       0.39     1.00 0.57
## Math          0.93   0.94 0.85       0.41       0.57     0.57 1.00
```

Examine the Two Variable Sets

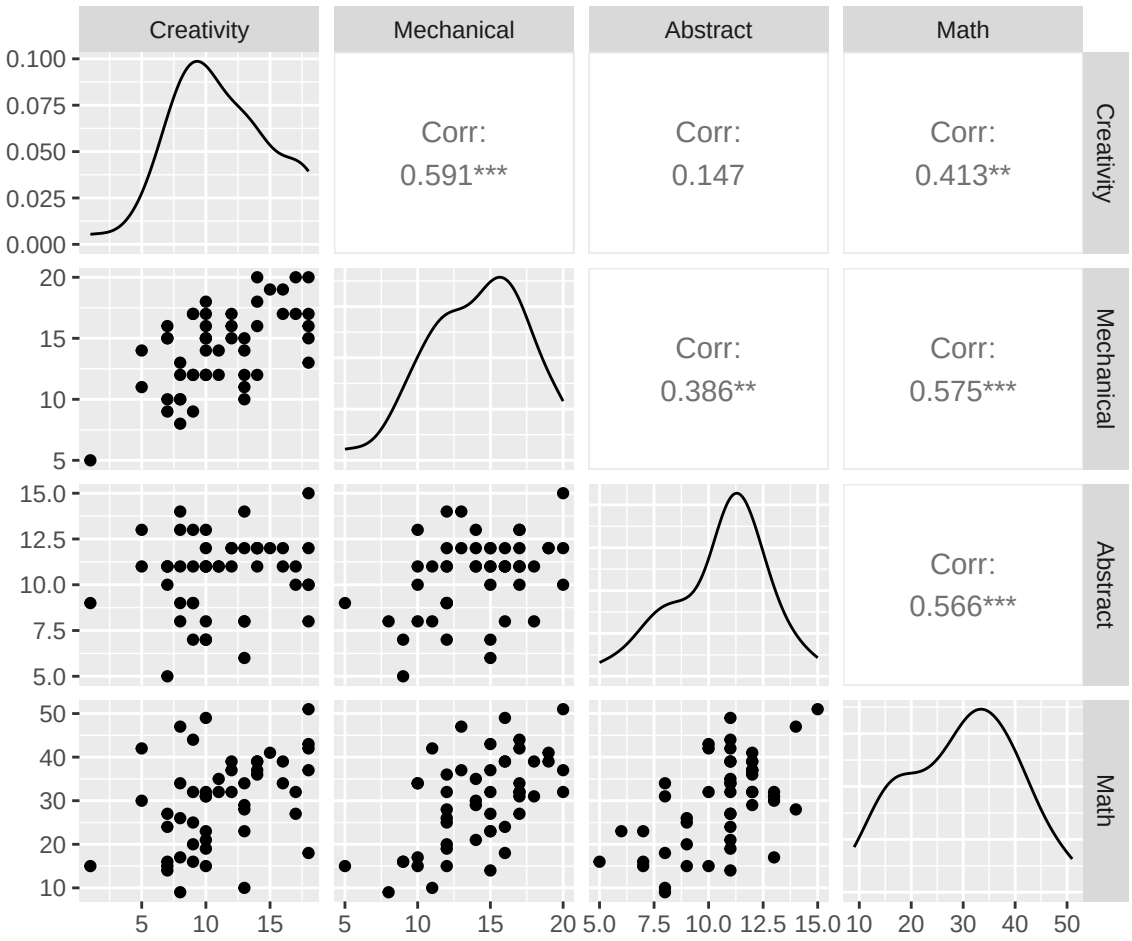
Separate the sales performance and intelligence variables.

```
sales <- sales_data[, 1:3]
intelligence <- sales_data[, 4:7]

ggpairs(sales)
```




```
ggpairs(intelligence)
```




```
# Correlations within and between the two sets
cor_mat <- matcor(sales, intelligence)

round(cor_mat$Xcor, 2)
```

```
##        Growth Profit  New
## Growth   1.00   0.93 0.88
## Profit   0.93   1.00 0.84
## New      0.88   0.84 1.00
```

```
round(cor_mat$Ycor, 2)
```

```
##            Creativity Mechanical Abstract Math
## Creativity       1.00       0.59     0.15 0.41
## Mechanical       0.59       1.00     0.39 0.57
## Abstract         0.15       0.39     1.00 0.57
## Math             0.41       0.57     0.57 1.00
```

```
round(cor_mat$XYcor, 2)
```

```
##            Growth Profit  New Creativity Mechanical Abstract Math
## Growth       1.00   0.93 0.88       0.57       0.71     0.67 0.93
```

```
## Profit       0.93   1.00 0.84       0.54       0.75     0.47 0.94
## New          0.88   0.84 1.00       0.70       0.64     0.64 0.85
## Creativity   0.57   0.54 0.70       1.00       0.59     0.15 0.41
## Mechanical   0.71   0.75 0.64       0.59       1.00     0.39 0.57
## Abstract     0.67   0.47 0.64       0.15       0.39     1.00 0.57
## Math         0.93   0.94 0.85       0.41       0.57     0.57 1.00
```

Canonical Correlation Analysis

CCA constructs linear combinations of the two variable sets:

$$U_k = a_k^T X, \qquad V_k = b_k^T Y,$$

where each pair (U_k, V_k) maximizes the correlation subject to the canonical constraints.

Since the smaller set contains three variables, there are three canonical correlations.

```
cca_fit <- cc(sales, intelligence)

# Canonical correlations
cca_fit$cor
```

```
## [1] 0.9944827 0.8781065 0.3836057
```

```
# Canonical coefficient vectors
cca_fit$xcoef
```

```
##               [,1]       [,2]       [,3]
## Growth -0.06237788 -0.1740703  0.3771529
## Profit -0.02092564  0.2421641 -0.1035150
## New    -0.07825817 -0.2382940 -0.3834151
```

```
cca_fit$ycoef
```

```
##                   [,1]        [,2]        [,3]
## Creativity -0.06974814 -0.19239132 -0.24655659
## Mechanical -0.03073830  0.20157438  0.14189528
## Abstract   -0.08956418 -0.49576326  0.28022405
## Math       -0.06282997  0.06831607 -0.01133259
```

The coefficients define the canonical variates. Their magnitudes depend on the measurement scales of the original variables.

Test the Canonical Correlations

We test whether the two sets are uncorrelated:

$$H_0 : \Sigma_{XY} = 0.$$

Wilks' lambda provides a sequential test of the canonical correlations using an approximate F distribution.

```r
n <- nrow(sales)
p <- ncol(sales)
q <- ncol(intelligence)

p.asym(cca_fit$cor, n, p, q, tstat = "Wilks")
```

```
## Wilks' Lambda, using F-approximation (Rao's F):
##                 stat    approx df1      df2      p.value
## 1 to 3:  0.002148472 87.391525  12 114.0588 0.000000e+00
## 2 to 3:  0.195241267 18.526265   6  88.0000 8.248957e-14
## 3 to 3:  0.852846693  3.882233   2  45.0000 2.783536e-02
```

Interpretation: The first test assesses whether all canonical correlations are zero. Subsequent tests assess the remaining canonical correlations after removing the preceding pairs.

Canonical Scores and Loadings

The canonical scores are the observed values of the canonical variates. Canonical loadings measure the correlations between the original variables and the canonical variates.

```r
# Canonical scores
cca_fit$scores
```

```
## $xscores
##              [,1]         [,2]        [,3]
##  [1,]  0.97838292 -0.362539552  0.81938141
##  [2,]  1.40651588 -0.410239408  0.05351720
##  [3,]  0.66973709  0.044672581  0.66847466
##  [4,] -0.40689705 -2.063089470 -0.30840196
##  [5,] -0.23688307 -0.310765017  0.99852234
##  [6,]  0.65494914 -0.844131320  1.14501451
##  [7,]  0.65528867 -0.236093843  0.93986313
##  [8,] -2.04552806 -1.334870222 -1.86845037
##  [9,] -0.35985473 -0.519574441  0.94175512
## [10,] -0.72379436  2.167475467  1.87763089
## [11,] -0.43808377 -0.291022268  0.89837606
## [12,]  0.04665613  1.736460981  0.67680067
## [13,] -0.74183310 -0.386875380 -0.45592909
## [14,]  0.02197133 -0.265760835  0.56211507
## [15,] -0.07973516 -0.108696870  0.40371149
## [16,]  1.96716552  1.701082028 -2.18552760
## [17,] -0.12525281 -0.746667218  1.06998582
## [18,] -0.41988032  0.070288835  0.98656859
## [19,]  0.25428846  0.007931722 -1.16609979
## [20,] -0.28687624 -1.267369772 -0.95084222
## [21,]  1.43024767  0.727850022 -0.03851613
## [22,] -0.32086304  1.593024005 -1.26353824
## [23,]  1.55121735  0.223860881 -0.12582704
## [24,] -0.75246285  0.068144053  0.71947930
## [25,] -1.29453998  0.585685240 -0.78260154
## [26,]  0.83411571  1.038223887  0.31143725
```

```
## [27,] -1.08624858  0.299106887  0.10431852
## [28,] -0.93245443  1.379170139  0.85582619
## [29,]  0.97434443 -1.976531657  0.32682226
## [30,] -0.89871368  1.779857589  0.67875923
## [31,] -1.31816090  0.039068369 -1.57960457
## [32,]  1.41677918 -0.081040167 -0.23857427
## [33,]  0.42719503  0.239653495  0.15995996
## [34,]  0.83477033 -1.238029918  1.00485465
## [35,] -1.39120112  0.436361392 -1.59805812
## [36,] -0.99174366 -0.182594033  0.36498468
## [37,]  0.52144946 -0.699170907 -2.10553997
## [38,] -0.09295538 -2.319565933  1.71161788
## [39,] -1.36370547  0.793354173 -0.23516470
## [40,] -1.06804514  0.660417991  0.19251105
## [41,] -0.36206701 -0.225189261  0.72509423
## [42,]  0.75616793  0.941121130 -0.61716522
## [43,] -0.70972028  0.097445954 -0.77782264
## [44,]  1.88288869 -0.423444109 -1.36329886
## [45,]  0.58821767  0.252097550  0.13103950
## [46,] -1.02875134 -1.413474738 -1.86114887
## [47,]  1.23583458  0.365853533 -0.30894632
## [48,]  1.92471305  0.484483866 -0.47810330
## [49,] -0.68982939 -1.133476875  0.34804581
## [50,] -0.86681530  1.107521445  0.63269334
##
## $yscores
##              [,1]        [,2]        [,3]
##  [1,]  0.97479103  0.09430244 -0.08851950
##  [2,]  1.40034960 -0.76140727  0.45769014
##  [3,]  0.66755933  0.69659017  0.09004153
##  [4,] -0.19984043 -1.14455925 -0.05227647
##  [5,] -0.20982423 -0.16086269  0.79529079
##  [6,]  0.60160796 -0.61815056  0.49782999
##  [7,]  0.66064116  0.43588278 -0.14518246
##  [8,] -2.38396289 -0.88140585  0.15766738
##  [9,] -0.29803503 -0.32179325  1.37063799
## [10,] -0.93127733  0.64827089 -0.12480193
## [11,] -0.41079711 -0.14249657  0.58596818
## [12,]  0.11904755  2.35859742  0.11141301
## [13,] -0.72588545 -0.27966647 -0.70314741
## [14,]  0.04726778 -0.15155656  0.27603833
## [15,] -0.03278039  0.37377660 -1.79741676
## [16,]  1.81607881  1.58415070 -1.09665800
## [17,] -0.09779329 -0.46221390 -0.15717568
## [18,] -0.34775982  0.14588306  0.09261709
## [19,]  0.20575055 -0.03287585  2.18906768
## [20,] -0.35582377 -0.95027027 -0.87037584
## [21,]  1.39832111  0.55185730 -0.83886122
## [22,] -0.27686589  1.17372033  1.06694262
## [23,]  1.49745418  0.20784154 -1.02932306
## [24,] -0.66347038 -0.20406500  0.24551465
## [25,] -1.19107609 -0.03070063 -0.19579577
## [26,]  0.80346642  1.70310899 -0.50383613
## [27,] -0.95936491 -0.25064113 -0.12836843
```

```
## [28,] -1.21910791  1.69784810  0.46430792
## [29,]  0.93624898 -2.30445623  1.02914053
## [30,] -0.86594820  1.75023346  0.90942276
## [31,] -1.27981072  0.04300999 -1.86226853
## [32,]  1.44440068 -1.06423465 -1.38353924
## [33,]  0.40313424  0.57049419  1.31139946
## [34,]  0.78874451 -1.15793146  0.23670462
## [35,] -1.27845734  0.37784268 -1.56714538
## [36,] -1.13042931 -0.14601431  1.39507261
## [37,]  0.43932868 -0.47179081 -1.99750652
## [38,] -0.25466221 -2.60755059  0.23571368
## [39,] -1.24698790  0.29832283  0.02809563
## [40,] -0.95665816  0.41902427  0.46187787
## [41,] -0.29111687 -0.06108586  1.60586199
## [42,]  0.68378617  1.62169828 -1.52372994
## [43,] -0.67262436  0.49179647 -0.78157702
## [44,]  2.06209412 -0.11916799  0.94732921
## [45,]  0.56088586  0.56712037  1.48729252
## [46,] -1.02048263 -1.76156170 -1.51761542
## [47,]  1.21992390 -0.31761469  1.45872318
## [48,]  1.94818626 -0.77531725 -0.56510955
## [49,] -0.64792179 -1.26188686 -0.66195176
## [50,] -0.73030445  0.62990478  0.08452069
##
## $corr.X.xscores
##              [,1]          [,2]         [,3]
## Growth -0.9798776  0.0006477883  0.199598477
## Profit -0.9464085  0.3228847489 -0.007504408
## New    -0.9518620 -0.1863009724 -0.243414776
##
## $corr.Y.xscores
##                  [,1]       [,2]        [,3]
## Creativity -0.6348095 -0.1894059 -0.24988439
## Mechanical -0.7171837  0.2086069  0.02598458
## Abstract   -0.6436782 -0.4402237  0.22027544
## Math       -0.9388771  0.1734549  0.03614570
##
## $corr.X.yscores
##              [,1]          [,2]         [,3]
## Growth -0.9744713  0.0005688272  0.076567107
## Profit -0.9411869  0.2835272081 -0.002878734
## New    -0.9466102 -0.1635921013 -0.093375287
##
## $corr.Y.yscores
##                  [,1]       [,2]        [,3]
## Creativity -0.6383313 -0.2156981 -0.65140953
## Mechanical -0.7211626  0.2375644  0.06773775
## Abstract   -0.6472493 -0.5013329  0.57422365
## Math       -0.9440859  0.1975329  0.09422619
```

```
# Correlations between variables and canonical variates
cca_load <- comput(sales, intelligence, cca_fit)

# Within-set loadings
```

```
cca_load$corr.X.xscores
```

```
##              [,1]          [,2]         [,3]
## Growth -0.9798776  0.0006477883  0.199598477
## Profit -0.9464085  0.3228847489 -0.007504408
## New    -0.9518620 -0.1863009724 -0.243414776
```

```
cca_load$corr.Y.yscores
```

```
##                  [,1]       [,2]        [,3]
## Creativity -0.6383313 -0.2156981 -0.65140953
## Mechanical -0.7211626  0.2375644  0.06773775
## Abstract   -0.6472493 -0.5013329  0.57422365
## Math       -0.9440859  0.1975329  0.09422619
```

```
# Cross-loadings
cca_load$corr.X.yscores
```

```
##              [,1]          [,2]         [,3]
## Growth -0.9744713  0.0005688272  0.076567107
## Profit -0.9411869  0.2835272081 -0.002878734
## New    -0.9466102 -0.1635921013 -0.093375287
```

```
cca_load$corr.Y.xscores
```

```
##                  [,1]       [,2]        [,3]
## Creativity -0.6348095 -0.1894059 -0.24988439
## Mechanical -0.7171837  0.2086069  0.02598458
## Abstract   -0.6436782 -0.4402237  0.22027544
## Math       -0.9388771  0.1734549  0.03614570
```

Interpretation:

- **Loadings:** Correlations between variables and canonical variates from the same set.
- **Cross-loadings:** Correlations between variables and canonical variates from the other set.

Large absolute loadings help identify which original variables contribute most strongly to the interpretation of each canonical pair.

Verify CCA Using Eigenvalue Decomposition

The squared canonical correlations are the eigenvalues of

$$A = S_X^{-1/2} S_{XY} S_Y^{-1} S_{YX} S_X^{-1/2}$$

or, equivalently, the nonzero eigenvalues of

$$B = S_Y^{-1/2} S_{YX} S_X^{-1} S_{XY} S_Y^{-1/2}.$$

Here, the covariance matrices are estimated from the sample.

```r
# Sample covariance matrices
Sx <- cov(sales)
Sy <- cov(intelligence)
Sxy <- cov(sales, intelligence)

# Symmetric inverse square roots
Sx_invhalf <- solve(sqrtm(Sx))
Sy_invhalf <- solve(sqrtm(Sy))

# Eigenvalue matrices
A <- Sx_invhalf %*% Sxy %*% solve(Sy) %*%
     t(Sxy) %*% Sx_invhalf

B <- Sy_invhalf %*% t(Sxy) %*% solve(Sx) %*%
     Sxy %*% Sy_invhalf

# Compare eigenvalues with squared canonical correlations
eigen(A)$values
```

```
## [1] 0.9889958 0.7710711 0.1471533
```

```r
eigen(B)$values
```

```
## [1] 9.889958e-01 7.710711e-01 1.471533e-01 1.110223e-15
```

```r
cca_fit$cor^2
```

```
## [1] 0.9889958 0.7710711 0.1471533
```

The eigenvectors can also be transformed into canonical coefficient vectors:

$$a_k = S_X^{-1/2} u_k, \qquad b_k = S_Y^{-1/2} v_k.$$

```r
# Eigenvector-based canonical coefficients
a <- Sx_invhalf %*% eigen(A)$vectors[, 1]
b <- Sy_invhalf %*% eigen(B)$vectors[, 1]

# Compare with CCA output
cbind(Eigen = a, CCA = cca_fit$xcoef[, 1])
```

```
##                            CCA
## Growth -0.06237788 -0.06237788
## Profit -0.02092564 -0.02092564
## New    -0.07825817 -0.07825817
```

```r
cbind(Eigen = b, CCA = cca_fit$ycoef[, 1])
```

```
##                              CCA
## Creativity 0.06974814 -0.06974814
## Mechanical 0.03073830 -0.03073830
## Abstract   0.08956418 -0.08956418
## Math       0.06282997 -0.06282997
```

Note: Eigenvectors are determined only up to sign. Thus, the coefficients may differ by a sign even when they represent the same canonical variate.

Verify the Canonical Loadings

For the first canonical variate $U_1 = a_1^T X$, its loading for X_j is

$$\text{Corr}(X_j, U_1) = \frac{(S_X a_1)_j}{\sqrt{(S_X)_{jj}}\sqrt{a_1^T S_X a_1}}.$$

An analogous expression applies to $V_1 = b_1^T Y$.

```
# First canonical coefficient vectors
a1 <- cca_fit$xcoef[, 1]
b1 <- cca_fit$ycoef[, 1]

# Compute loadings from covariance matrices
load_x <- as.vector(Sx %*% a1) /
  (sqrt(diag(Sx)) * sqrt(drop(t(a1) %*% Sx %*% a1)))

load_y <- as.vector(Sy %*% b1) /
  (sqrt(diag(Sy)) * sqrt(drop(t(b1) %*% Sy %*% b1)))

# Compare with CCA loadings
cbind(Manual = load_x, CCA = cca_load$corr.X.xscores[, 1])
```

```
##            Manual        CCA
## Growth -0.9798776 -0.9798776
## Profit -0.9464085 -0.9464085
## New    -0.9518620 -0.9518620
```

```
cbind(Manual = load_y, CCA = cca_load$corr.Y.yscores[, 1])
```

```
##                Manual        CCA
## Creativity -0.6383313 -0.6383313
## Mechanical -0.7211626 -0.7211626
## Abstract   -0.6472493 -0.6472493
## Math       -0.9440859 -0.9440859
```

The manually calculated loadings should agree with the results from `comput()`, up to numerical precision.